



1. Problem Title & Link

Title: LeetCode 611 : Valid Triangle Number

Link: [LeetCode 611 - Valid Triangle Number](#)

2. Problem Statement (Short Summary)

You are given an integer array:

nums[]

Return the number of triplets that can form a valid triangle.

Triangle condition:

For three sides:

a,b,c

A triangle is valid if:

$a+b>c$

(In a sorted array, checking one condition is sufficient.)

In simple terms:

Count how many groups of three numbers can form a triangle.

3. Examples (Input → Output)

Example 1

Input:

nums=[2,2,3,4]

Output:

3

Explanation:

Valid triangles:

(2,3,4)

(2,3,4)

(2,2,3)

Example 2

Input:

nums=[4,2,3,4]

Output:

4

**Example 3**

Input:

nums=[1,1,3,4]

Output:

0

4. Constraints

$1 \leq \text{nums.length} \leq 1000$

$0 \leq \text{nums}[i] \leq 1000$

Special restrictions:

- Need count of all possible triangles
- Avoid $O(n^3)$

5. Core Concept (Pattern / Topic)

Sorting + Two Pointers

6. Thought Process (Step-by-Step Explanation)**Brute Force**

Generate every triplet:

1. Select three numbers
2. Check triangle condition
3. Count valid combinations

Time:

$O(n^3)$

Too slow.

Optimized Thinking

Observation:

After sorting:

$a \leq b \leq c$

Triangle condition becomes:

$a + b > c$

Only one condition needed.

Algorithm:

1. Sort array
2. Fix largest side c
3. Use two pointers:



left=0

right=i-1

If:

nums[left]+nums[right]

>

nums[i]

Then:

all values between

left...right-1

also work.

Count:

right-left

Move:

right--

Otherwise:

left++

7. Visual / Intuition Diagram (ASCII Diagram)

Input:

[2,2,3,4]

Sorted:

[2,2,3,4]

Fix:

c=4

Check:

2+3>4

true

Count:

right-left

=2

Triangles:

(2,3,4)

(2,3,4)



8. Pseudocode (Language Independent)

```
sort(nums)
```

```
count=0
```

```
for i=n-1 to 2:
```

```
    left=0
```

```
    right=i-1
```

```
    while left<right:
```

```
        if nums[left]
```

```
            +
```

```
            nums[right]
```

```
            >
```

```
            nums[i]:
```

```
                count +=
```

```
                right-left
```

```
                right--
```

```
        else:
```

```
            left++
```

```
return count
```

9. Code Implementation

Python

```
class Solution:
```

```
    def triangleNumber(
```

```
        self,
```



```
    nums):

nums.sort()

count=0

for i in range(
    len(nums)-1,
    1,
    -1
):

    left=0
    right=i-1

    while left<right:

        if(
            nums[left]
            +
            nums[right]
            >
            nums[i]
        ):

            count += (
                right-left
            )

            right-=1

        else:

            left+=1

    return count
```

**Java**

```
class Solution {

    public int triangleNumber(
        int[] nums) {

        Arrays.sort(nums);

        int count=0;

        for(
            int i=
                nums.length-1;
            i>=2;
            i--
        ){

            int left=0;
            int right=i-1;

            while(
                left<right
            ){

                if(
                    nums[left]
                    +
                    nums[right]
                    >
                    nums[i]
                ){

                    count+=
                        (
                            right-left
                        );
                }
            }
        }
    }
}
```



```
        right--;\n    }\n\n    else{\n\n        left++;\n    }\n}\n}\n\nreturn count;\n}\n}
```

10. Time & Space Complexity

Time Complexity:

$O(n^2)$

Reason:

- Sorting:

$O(n \log n)$

- Two pointer scan:

$O(n^2)$

Overall:

$O(n^2)$

Space Complexity:

$O(1)$

Ignoring sorting space.

11. Common Mistakes / Edge Cases

1. Checking all three triangle conditions

Wrong:

$a+b>c$

$a+c>b$

$b+c>a$



After sorting:

only one needed

2. Forgetting sorting

Two-pointer logic fails.

3. Using:

`count++`

instead of:

`count+=(right-left)`

4. Missing:

`[0,0,0]`

Output:

0

12. Detailed Dry Run (Step-by-Step Table)

Input:

`nums=[2,2,3,4]`

Sorted:

`[2,2,3,4]`

Largest Side	Left	Right	Condition	Count
4	0	2	$2+3>4$	2
3	0	1	$2+2>3$	1

Total:

3

13. Common Use Cases (Real-Life / Interview)

Geometry processing

Network topology validation

Structural design systems

Game physics engines

14. Common Traps (Important!)

1. Missing sorting

2. Counting only one triangle



3. Using $O(n^3)$
4. Incorrect pointer movement

15. Builds To (Related LeetCode Problems)

Progression:

- LC 611 → Valid Triangle Number
- LC 15 → 3Sum
- LC 16 → 3Sum Closest
- LC 18 → 4Sum
- LC 167 → Two Sum II

16. Alternate Approaches + Comparison

Approach	Time	Space	Notes
Brute Force	$O(n^3)$	$O(1)$	Too slow
Binary Search	$O(n^2 \log n)$	$O(1)$	Good
Sorting + Two Pointers	$O(n^2)$	$O(1)$	Optimal

17. Why This Solution Works (Short Intuition)

After sorting, if:

$\text{nums}[\text{left}] + \text{nums}[\text{right}]$

>

$\text{nums}[\text{i}]$

then every element between left and right also satisfies the condition because values are sorted. This allows counting multiple triangles at once.

18. Variations / Follow-Up Questions

1. Return actual triangle triplets
2. Find largest possible triangle perimeter
3. Count triangles with unique values
4. Support dynamic insertion of sides
5. Solve for 4-sided polygon validity
- 6.